

Physical AI Homework 3:

A Robot Manipulation Framework

111550132 張家睿

1 About task 1

1.1 Briefly explain how you implement your_fk() function

```
def your_fk(DH_params : dict, q : list or tuple or np.ndarray, base_pos) -> np.ndarray:

    # robot initial position
    base_pose = list(base_pos) + [0, 0, 0]

    assert len(DH_params) == 6 and len(q) == 6, f'Both DH_params and q should contain 6 values,\n' \
        f'but get len(DH_params) = {len(DH_params)}, len(q) = {len(q)}'

    A = get_matrix_from_pose(base_pose) # a 4x4 matrix, type should be np.ndarray
    jacobian = np.zeros((6, 6)) # a 6x6 matrix, type should be np.ndarray

    # ----- #
    # --- TODO: Read the task description --- #
    # --- Task 1 : Compute Forward-Kinematic and Jacobain of the robot by yourself --- #
    # --- Try to implement `your_fk` function without using any pybullet --- #
    # --- API. (20% for accuracy) --- #
    # ----- #

    #### your code ####

    def dh_transform(a, d, alpha, theta):
        ca, sa = np.cos(alpha), np.sin(alpha)
        ct, st = np.cos(theta), np.sin(theta)
        T = np.array([
            [ ct, -st*ca, st*sa, a*ct],
            [ st, ct*ca, -ct*sa, a*st],
            [ 0, sa, ca, d],
            [ 0, 0, 0, 1]
        ], dtype=np.float64)
        return T

    # Forward kinematics
    T_list = [A]
    T = A.copy()
    for i in range(6):
        a = DH_params[i]['a']
        d = DH_params[i]['d']
        alpha = DH_params[i]['alpha']
        theta = float(q[i])
        T = T @ dh_transform(a, d, alpha, theta)
        T_list.append(T)

    # End-effector pose
    A = T_list[-1].copy()

    # Jacobian
    p_e = T_list[-1][:3, 3]
    for i in range(6):
        z_im1 = T_list[i][:3, 2]
        p_im1 = T_list[i][:3, 3]
        jacobian[:3, i] = np.cross(z_im1, (p_e - p_im1))
        jacobian[3:, i] = z_im1

    # hint :
    # https://automaticaddison.com/the-ultimate-guide-to-jacobian-matrices-for-robotics/

    #####

    # adjustment don't touch
    adjustment = np.asarray([[ 0, -1, 0],
                             [ 0, 0, 0],
                             [ 0, 0, -1]])
    A[:3, :3] = A[:3, :3]@adjustment
    pose_7d = np.asarray(get_pose_from_matrix(A, 7))

    return pose_7d, jacobian
```

Implement `your_fk()` by chaining classic Denavit–Hartenberg transforms from the world-to-base pose to the end-effector, then building the geometric Jacobian in the world frame. First, form T_{w0} from the given base position. For each of the 6 revolute joints i right-multiply the classic DH link transform $T_i = R_z(\theta_i)T_z(d_i)T_x(a_i)R_x(\alpha_i)$, accumulating a list of world-to-frame matrices. The final transform $A = T_6$ gives the end-effector pose. Then apply the fixed alignment matrix provided by the assignment to $A_{0:3,0:3}$ so the tool frame matches the grader’s convention, and convert A to a 7D pose $[x, y, z, q_x, q_y, q_z, q_w]$. For the Jacobian, using geometric formulas for revolute joints expressed in the world frame, each column i is $J_{v,i} = z_{i-1} \times (p_{ee} - p_{i-1})$ and $J_{w,i} = z_{i-1}$, where z_{i-1} and p_{i-1} come from T_{i-1} and p_{ee} is the end-effector position. The function returns the 7D pose and the 6×6 Jacobian.

1.2 What is the difference between D-H convention and Craig’s convention

(Modified D-H Conveition)?

Classic DH and Craig’s Modified DH differ in frame assignment and parameter indexing. In classic DH, each link’s parameters $(a_i, \alpha_i, d_i, \theta_i)$ are defined with respect to frame i , and the link transform is ordered $R_z(\theta_i)T_z(d_i)T_x(a_i)R_x(\alpha_i)$. In Craig’s modified DH, the link parameters (a_{i-1}, α_{i-1}) are associated with the previous frame $i-1$, and the transform is ordered $R_x(\alpha_{i-1})T_x(a_{i-1})R_z(\theta_i)T_z(d_i)$. So the intermediate frames and the DH tables are different and not interchangeable, even though both conventions produce the same physical end-effector pose when applied consistently.

1.3 Complete the D-H table in your report following D-H convention

i	d	α (rad)	a	θ_i (rad)
1	d1	$\pi / 2$	0	θ_1
2	0	$-\pi / 2$	0	θ_2
3	d3	$\pi / 2$	a3	θ_3
4	0	$-\pi / 2$	a4	θ_4
5	d5	$\pi / 2$	0	θ_5
6	0	$\pi / 2$	a6	θ_6
7	d7	0	0	θ_7

2 About task 2

2.1 Briefly explain how you implement `your_ik()` function

```
def your_ik(robot_id, new_pose : list or tuple or np.ndarray,
            base_pos, max_iters : int=1000, stop_thresh : float=.001):

    joint_limits = np.asarray([
        [-3*np.pi/2, -np.pi/2], # joint1
        [-2.3562, -1],          # joint2
        [-17, 17],              # joint3
        [-17, 17],              # joint4
        [-17, 17],              # joint5
        [-17, 17],              # joint6
    ])

    # get current joint angles and gripper pos, (grripper pos is fixed)
    num_q = p.getNumJoints(robot_id)
    q_states = p.getJointStates(robot_id, range(0, num_q))

    tmp_q = np.asarray([x[0] for x in q_states][2:8]) # current joint angles 6d (You only need to modify this)
```

```

# Algorithm choose
Algorithm = "Pseudo-Inverse" # "Damped Least Squares" # BONUS

# Target position and quaternion
tgt_pos = np.asarray(new_pose[:3], dtype=np.float64)
tgt_quat = np.asarray(new_pose[3:7], dtype=np.float64)
if np.linalg.norm(tgt_quat) == 0:
    tgt_quat = np.array([0.0, 0.0, 0.0, 1.0])
else:
    tgt_quat = tgt_quat / np.linalg.norm(tgt_quat)

# Hyper parameters
alpha = 0.5
damping = 1e-3
w_pos = 1.0
w_rot = 0.5
dq_cap = 0.3
rcond = 1e-4

DH = get_ur5_DH_params()

for _ in range(max_iters):
    cur_pose, J = your_fk(DH, tmp_q, base_pos)
    cur_pos = np.asarray(cur_pose[:3], dtype=np.float64)
    cur_quat = np.asarray(cur_pose[3:7], dtype=np.float64)
    if np.linalg.norm(cur_quat) == 0:
        cur_quat = np.array([0.0, 0.0, 0.0, 1.0])
    else:
        cur_quat = cur_quat / np.linalg.norm(cur_quat)

    # Position error
    pos_err = tgt_pos - cur_pos
    # Rotation error
    Rc = R.from_quat(cur_quat).as_matrix()
    Rd = R.from_quat(tgt_quat).as_matrix()
    R_err = Rc.T @ Rd
    rot_err = R.from_matrix(R_err).as_rotvec()

    dx = np.hstack([w_pos * pos_err, w_rot * rot_err])
    if np.linalg.norm(dx) < stop_thresh:
        break

    if Algorithm == "Pseudo-Inverse":
        # Pseudo-inverse
        dq = alpha * (np.linalg.pinv(J, rcond=rcond) @ dx)
    else: # BONUS
        # Damped Least Squares (Levenberg-Marquardt method)
        # https://en.wikipedia.org/wiki/Levenberg%E2%80%93Marquardt_algorithm
        dq = alpha * (J.T @ np.linalg.inv(J @ J.T + (damping**2) * np.eye(6)) @ dx)
    dq = np.clip(dq, -dq_cap, dq_cap)

    # Update joint angles
    tmp_q = tmp_q + dq
    for j in range(6):
        lo, hi = joint_limits[j]
        tmp_q[j] = np.clip(tmp_q[j], lo, hi)

return list(tmp_q) # 6 DoF

```

Initializes the current 6-DoF arm configuration by reading joints from PyBullet, parses the target pose into position and a normalized quaternion, and sets step size, weighting, and safety caps. Next, in each iteration, it calls your own `your_fk()` to get the current end-effector pose and the 6x6 geometric Jacobian, forms a 6D space error by combining translational error with an orientation error computed as the rotation vector of $R_c^T R_d$, and, if that error is above the threshold, computes a joint update either with a Pseudo-inverse $\Delta q = \alpha \text{pinv}(J) dx$ or, in the bonus branch, with damped least squares $\Delta q = \alpha J^T (J J^T + \lambda^2 I)^{-1} dx$. The update is clipped to avoid overshoot, applied to the joint vector, and hard clamped to joint limits, repeating until the error is small enough or the iteration budget is exhausted, at which point it returns the converged solution.

2.2 What problems do you encounter and how do you deal with them?

```

(pdm-hw3) cjchang@hcis-s28:~/physical-ai25/hw3$ python ik.py
pybullet build time: Sep 22 2020 00:55:20
startThreads creating 1 threads.
starting thread 0
started thread 0
argc=2
argv[0] = --unused
argv[1] = --start_demo_name=Physics Server
ExampleBrowserThreadFunc started
X11 functions dynamically loaded using dlopen/dlsym OK!
X11 functions dynamically loaded using dlopen/dlsym OK!
libGL error: MESA-LOADER: failed to open swrast: /usr/lib/dri/swrast_dri.so: cannot open shar
i, suffix_dri)
libGL error: failed to load driver: swrast
Creating context
Failed to create GL 3.3 context ... using old-style GLX context
Failed to create an OpenGL context
(pdm-hw3) cjchang@hcis-s28:~/physical-ai25/hw3$

```

I use ssh to connect to server with x11-forwarding. Normally it can shows GUI on the size of my monitor. However, it always shows up openGL issue. I solve it by replacing X11 ssh (MobaXterm) with Remote Desktop (NoMachine).

2.3 Bonus! Do you also implement other IK methods instead of pseudo inverse method? How about the results?

Yes, I implement Damped Least Squares(Levenberg-Marquardt method). You can change the algorithm by changing the 88th line .

```
87 | # Algorithm choose
88 | Algorithm = "Pseudo-Inverse" # "Damped Least Squares" # BONUS
```

Damped Least Squares is more stable and numerically more resistant to near singularities.

3 About task 3

3.1 Compare your results between your_ik function and pybullet_ik

Both your_ik() and pybullet_ik() can get full point in task 3. pybullet_ik() might be a little bit faster.

```
===== Task 3 : Transporter Network =====
Test: 1/10
WARNING:tensorflow:From /home/hcis-s15/louis-chang/Physical-AI-course/hw3/ravens/ravens/agents/transporter.py:167: set_learning_phase (from tensorflow
.python.keras.backend) is deprecated and will be removed after 2020-10-11.
Instructions for updating:
Simply pass a True/False value to the 'training' argument of the '__call__' method of your layer or model.
W1111 19:46:41.672663 139653071463488 deprecation.py:323] From /home/hcis-s15/louis-chang/Physical-AI-course/hw3/ravens/ravens/agents/transporter.py:1
67: set_learning_phase (from tensorflow.python.keras.backend) is deprecated and will be removed after 2020-10-11.
Instructions for updating:
Simply pass a True/False value to the 'training' argument of the '__call__' method of your layer or model.
Total Reward: 1.0 Done: True
Test: 2/10
Total Reward: 1.0 Done: True
Test: 3/10
Total Reward: 1.0 Done: True
Test: 4/10
Total Reward: 1.0 Done: True
Test: 5/10
Total Reward: 1.0 Done: True
Test: 6/10
Total Reward: 1.0 Done: True
Test: 7/10
Total Reward: 1.0 Done: True
Test: 8/10
Total Reward: 1.0 Done: True
Test: 9/10
Total Reward: 1.0 Done: True
Test: 10/10
Total Reward: 1.0 Done: True
=====
- Your Total Score : 10.000 / 10.000
=====
```